

P0812R0: copy-list-initialization is inherently un- =

Audience: EWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

October 16, 2017

Problem

Explicit constructors protect against certain silly mistakes:

```
void useInts(std::unique_ptr<int[]>);

void f() {
    vector<int> v;
    // ...
    useInts(v.data()); // oops
}
```

The intent of the restriction to direct-initialization is that explicit constructors (or conversion functions) can be used only where the syntax indicates that (for class types) a constructor is being called:

```
f(T(v));           // explicit OK
f(v);              // not considered

T x(v);           // OK
T x=v;            // not considered
T x[]={v,v};      // not considered

struct A {
    T t;
    A() : t(v) {} // OK
};
```

Note that all these examples are C++03, which has several implications:

1. The forms with = always initialize with an expression, not an argument list.
2. More generally, copy-initialization is always converting a single value, either that being converted or (one of those) being "assigned" in an initializer. (So explicit was meaningful for a constructor only if it could be called with one argument.)
3. Only in the mem-initializer case does the syntax indicate object construction without having T textually present.

C++11 changed all of these:

1. `T x={u,v};` initializes with an argument list for non-aggregate T.
2. `explicit` applies to all constructors to support that case.
3. Function arguments (including "constructor calls", `operator[]`, and any assignment operator) and return values can be braced-init-lists which construct an object of an inferred type.

The prohibition ([over.match.list]/1) on the use of explicit constructors for this last case (copy-list-initialization without =) doesn't protect against anything:

```
std::unique_ptr<int[]> getInts() {
    const int count=/* ... */;
```

```

return {new int[count]}; // ill-formed
}

```

The braces indicate the programmer's intent to construct a new object. Since they prevent the direct use of conversion functions (which might also motivate their use), a constructor must be called. The type being constructed is as obvious as in the mem-initializer case: it is the return type of the function. Yet the constructor called must be non-explicit!

In C++03, direct-initialization usually involved the type name. C++11 followed that rule of thumb by treating a braced-init-list as if it were an expression to be converted despite it being syntax for object construction. It is telling that the Google C++ Style Guide [calls out](#) this extension of the reach of explicit.

Solution

Possibilities

When returning a braced-init-list, the type to be constructed is always known. Otherwise, the braced-init-list is a function argument and might be used to construct one of several types to be selected by overload resolution. The simplest change, then, would be to allow the use of any constructor only in the former case. However, consider

```

void f(std::unique_ptr<int[]>, std::unique_ptr<float[]>);

void g() {
    f(std::make_unique<int[]>(9), std::make_unique<float[]>(6));
    f({new int[9]}, {new float[6]}); // ill-formed
}

```

The simpler call to `f` is disallowed with no more benefit than in the return value case. In particular, C++17 made constructions like this safe by requiring that one of the `unique_ptr` parameters is initialized before the other `new` is evaluated.

The obvious approach is to make all list-initialization without `=` be direct-list-initialization, but that would adversely affect the single-element braced-init-lists which [dcl.init.list]/3.2 handles differently for copy-list-initialization.

Proposal

A less pervasive change is to limit, when there is no `=`, the prohibition on the use of an explicit constructor to cases where a non-explicit constructor is also viable. Then classes can continue to use `explicit` to mark confusing cases:

```

struct Score {
    explicit Score(const char*, std::size_t, double thresh=1);
    Score(std::string_view, double thresh=1);
};

Score s() {
    return {"foo", 2}; // error: Score(std::string_view, double) viable
}

```

Explicitness restricts the candidates for copy-initialization, but for copy-list-initialization merely makes it ill-formed to select one. Narrowing the scope of that ill-formed case is therefore a pure language extension that does not change the meaning of any well-formed program.

Limitations

Because types like `in_place_t` that define an explicit default constructor are particularly aiming to require `in_place` instead of just `{}` as the tag, the permission to use explicit constructors should not be casually extended to empty braced-init-lists.

Consider also this example which would become well-formed:

```
void f(std::string, float); // #1
void f(std::ofstream, int); // #2

void g() {f({"foo"}, 1);} // selects #2
```

With or without this change, other arguments usually determine which type to instantiate from a braced-init-list (since [over.ics.rank]/3.3 makes almost any two user-defined conversion sequences equivalent). If, however, the newly allowable use of the explicit constructor here is deemed overly surprising, it would be possible to prohibit the use of an explicit constructor for a braced-init-list in a function call where another viable function would not require an explicit constructor for it.

Wording

Based on N4687.

Change [class.conv.ctor]/2 as follows:

[Note: An explicit constructor constructs objects just like non-explicit constructors, but does so only where the ~~direct initialization syntax (11.6) or where casts (8.2.9, 8.4) are explicitly used~~ syntax explicitly constructs an object; see also 16.3.1.4.

Change [over.match.list]/1 as follows:

In copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed if a non-explicit constructor is also viable, if the initializer list has no elements, or if the initializer begins with = ([dcl.init]/17.1).